Universidad El Bosque

Facultad de Ingeniería — Programa de Ingeniería de Sistemas

Bases de Datos 1

Ing., Esp., M.Ed. Christian Felipe Duarte

Taller 1 — Reducción a Tablas: Reglas de Transformación del Modelo Entidad–Relación

Modelo de partida: Terminal Portuaria Bahía Verde

Juan David Polo Herrera — Sebastián Castellanos Parra — Juan Miguel Ramos

Bogotá D.C., 2026-2

1. Transformación de atributos

La idea en este caso es sencilla: en el modelo transformado no puede quedar ningún atributo compuesto, multivaluado ni derivado. Todos tienen que salir de alguna forma.

1a. Atributos de IMPORTADOR

IMPORTADOR viene con los tres problemas al tiempo, uno de cada tipo. Entonces toca ir uno por uno.

- **El compuesto: nombre_completo.** Este es el más sencillo. El atributo de arriba no guarda nada, solo está agrupando a *nombres* y *apellidos*. Entonces lo borramos, dejamos los dos hijos sueltos ahí en la entidad y queda resuelto.
- **El multivaluado: correo.** Este de esta forma sí toca trabajarlo. Un atributo no puede guardar varios valores, entonces sacamos *correo* de IMPORTADOR y lo volvemos una entidad aparte, CORREO_IMPORTADOR, pegada con una relación 1:N que le pusimos TENER. Así cada correo es su propia fila y el importador puede tener los que quiera.
- **El derivado: antiguedad_cliente.** Este se va y de esta forma. No se guarda porque se saca de *fecha_registro* con una resta.

CORREO_IMPORTADOR queda como entidad débil. O sea que no se sostiene sola: si borras el importador, sus correos de esta forma no significan nada. Y *correo* solito tampoco alcanza para saber cuál fila es cuál, apenas es clave parcial. La clave completa termina siendo **documento + correo**, y ese documento le llega desde IMPORTADOR por la relación TENER.

Cardinalidad y participación: IMPORTADOR queda en **(1) parcial**. Parcial porque puede haber un importador registrado que todavía no ha dado ningún correo. Y CORREO_IMPORTADOR queda en **(N) total**, total porque un correo suelto, sin dueño, no puede existir ahí.

Modelo: 01_Ej1a_IMPORTADOR.drawio

1b. El atributo sede de CONTRATISTA

Este está más enredado porque sede es multivaluado y compuesto al mismo tiempo. Y lo importante acá es que el orden en que apliques las reglas cambia el resultado, así que pilas con eso.

Nosotros lo hicimos así. Primero aplicamos la regla del multivaluado: sacamos sede completa de CONTRATISTA y la volvemos la entidad SEDE_CONTRATISTA, conectada con una relación 1:N que llamamos UBICAR. Después aplicamos la regla del compuesto: de esta forma con la entidad creada, partimos sede en ciudad y direccion.

¿Por qué no funciona hacerlo al contrario?

Si arrancas descomponiendo, te quedan dos multivaluados sueltos dentro de CONTRATISTA: ciudad por un lado y direccion por el otro. Y cuando los saques después, cada uno termina en su propia entidad y ahí se daña todo.

El problema es que el modelo deja de saber cuál dirección va con cuál ciudad. Por ejemplo, un contratista tiene sede en Bogotá (Calle 100) y en Cali (Av 6). Al separarlos, el modelo solo sabe que hay dos ciudades y dos direcciones, pero no cuál va con cuál. Te salen cuatro combinaciones posibles cuando en realidad solo eran dos. Por eso: **primero se extrae, después se descompone.**

La clave parcial de SEDE_CONTRATISTA es la pareja (ciudad, direccion), y la completa es **nit + ciudad + direccion**. La participación queda igual al punto de arriba: CONTRATISTA (1) parcial — SEDE_CONTRATISTA (N) total, porque puede haber contratista sin sede cargada, pero no una sede sin contratista.

Modelo: 02_Ej1b_CONTRATISTA_sede.drawio

1c. Por qué se van antiguedad_cliente y antiguedad

Los dos son atributos derivados, o sea que ese valor nadie lo escribe: se calcula. La antigüedad del cliente sale de *fecha_registro* y la del buque sale de *anio_construccion*. En los dos casos es una resta contra la fecha de hoy, nada más.

No se pueden dejar por dos cosas. La primera es que guardar algo que de esta forma puedes calcular es redundancia, y la redundancia siempre termina en datos que no cuadran: alguien actualiza un campo y se le olvida el otro. La segunda, que es la peor, es que ese valor se desactualiza solo. Mañana la antigüedad de esta forma cambió aunque nadie hade esta forma tocado nada, entonces te tocaría recalcular la tabla completa todos los días.

¿Y se pierde algo? No, nada. Los datos de base siguen ahí: *fecha_registro* y *anio_construccion* no se van, y esos sí son fijos. La antigüedad la sacas en el momento de la consulta y siempre te da el número bueno. Lo único que dejamos de hacer es guardarla.

1d. Si el importador solo puede tener dos correos

La regla como tal no cambia, un multivaluado sigue siendo multivaluado. Lo que pasa es que al ponerle un tope fijo aparece una segunda opción que antes no tenía ni sentido. Las dos sirven, y cuál escoges depende de qué tan firme sea esa regla.

Opción A: dejar la entidad como quedó en 1a, pero cambiando la cardinalidad de IMPORTADOR a (2) en vez de (N). Lo bueno: si mañana el puerto dice que ahora son tres correos, o que otra vez son ilimitados, solo cambias el número y no tocas nada más. Lo malo: estás manteniendo una entidad completa y una relación para guardar dos valores, que es harto para lo poquito que es.

Opción B: dos atributos simples, *correo1* y *correo2*, metidos directo en IMPORTADOR. Lo bueno: te ahorras una entidad, el modelo queda más corto y las consultas salen directas, sin joins. Lo malo: el límite queda clavado en la estructura, así que pasar a tres correos de esta forma no es cambiar un numerito sino tocar el esquema. Y para buscar quién tiene cierto correo toca mirar los dos campos.

Nosotros creemos que la opción B solo vale la pena si ese tope de dos es una regla del negocio de las que no se mueven. Si es una política administrativa que puede cambiar el otro semestre, mejor nos quedamos con la entidad débil.

2. Relaciones N:M con atributos

2a. La relación EMBARQUE

Una relación N:M no sobrevive al modelo transformado, así de simple. Y para rematar, esta trae atributos propios (*tipo_carga*, *peso_toneladas*, *fecha_solicitud*) que no son ni de IMPORTADOR ni de RECALADA.

Lo que hicimos fue volver esa relación una entidad, que le dejamos el mismo nombre EMBARQUE, y pegarla a cada lado con una relación 1:N. Los tres atributos se van para adentro de esa entidad nueva, que es donde de verdad van.

EMBARQUE queda como entidad débil porque no tiene nada propio con qué identificarse. Su identidad es literalmente la pareja: depende de IMPORTADOR y de RECALADA al tiempo. La clave le queda **documento + codigo_servicio + fecha_arribo**.

Ojo acá porque es donde se suele cometer el error: RECALADA **también** es débil, depende de SERVICIO. Entonces su identificador no es solo *fecha_arribo*, es la pareja (codigo_servicio, fecha_arribo). Por eso la clave de EMBARQUE termina con tres pedazos y no con dos. La debilidad se va encadenando.

La cardinalidad y la participación quedan así: IMPORTADOR (1) parcial — EMBARQUE (N) total, y RECALADA (1) parcial — EMBARQUE (N) total. El lado de la entidad débil siempre va total, porque un embarque sin importador o sin recalada no existe. Los lados fuertes van parciales, porque sí puede haber un importador que todavía no ha embarcado nada, o una recalada que llegó vacía.

Modelo: 03_Ej2a_EMBARQUE.drawio. Ahí también dibujamos SERVICIO – IDENTIFICAR – RECALADA, porque sin esa parte no se puede sustentar de dónde sale la clave.

2b. La relación CUADRILLA

Misma receta del punto de arriba. CUADRILLA se vuelve entidad débil con dos relaciones 1:N, una para OPERARIO y otra para RECALADA, y los atributos *turno* y *horas_trabajadas* se van para adentro. La clave queda **id_operario + codigo_servicio + fecha_arribo**, y la participación es OPERARIO (1) parcial — CUADRILLA (N) total; RECALADA (1) parcial — CUADRILLA (N) total.

Por qué turno y horas_trabajadas no corresponden en las entidades

Porque no describen al operario ni a la recalada por aparte. Describen el cruce de los dos. Solo cuando dices "este operario, en esta recalada" tiene sentido preguntar qué turno hizo y cuántas horas trabajó. Si te falta cualquiera de los dos, la pregunta no se puede responder.

Un ejemplo para verlo: el operario 101 trabaja el 12/05 en la recalada A, turno noche, 6 horas. Y el 13/05 en la recalada B, turno día, 8 horas.

- Si los metemos en OPERARIO, el 101 tiene un solo campo *turno* y un solo campo *horas_trabajadas*, entonces solo cabe uno de los dos días. El otro **se pierde**, no hay dónde meterlo.
- Si los metemos en RECALADA, la recalada A tendría un solo turno para toda la cuadrilla, así hade esta forman trabajado quince personas en horarios distintos. **Se pierde** la diferencia entre

operarios, y encima el dato se **duplica**, porque el mismo turno se repite para todos los que estuvieron ahí.

En cualquiera de los dos casos el modelo de esta forma no puede responder algo tan sencillo como "cuántas horas trabajó el operario 101 en la recalada A", que es justo el dato que el puerto necesita para pagar la nómina.

Modelo: 04_Ej2b_CUADRILLA.drawio

2c. Por qué esta transformación no cambia el significado

En el modelo original, cada pareja (importador, recalada) es un hecho único con sus datos. EMBARQUE representa exactamente esa pareja, ni más ni menos: como su clave es la unión de los dos identificadores, hay una fila por cada pareja real y no se puede repetir. Las dos relaciones 1:N vuelven a armar la N:M original sin inventarse combinaciones nuevas ni borrar las que de esta forma había.

En cambio, poner dos relaciones 1:N directas sí daña el modelo. Si pones IMPORTADOR (1) — (N) RECALADA, estás diciendo que cada recalada tiene un solo importador, pero el modelo original deja que varios importadores manden carga en el mismo barco. Con esa versión se pierden todos los embarques menos uno. Y si lo volteas, RECALADA (1) — (N) IMPORTADOR, ahora cada importador solo puede usar una recalada en toda su vida, que es igual de malo.

Y aparte de lo de la cardinalidad, en ninguna de las dos versiones hay un lugar decente para meter *tipo_carga*, *peso_toneladas* y *fecha_solicitud*, porque esos datos describen la pareja y no a ninguno de los dos por aparte. Donde los pongas van a quedar mal.

3. Relación ternaria y agregación

3a. La relación ternaria ABASTECER

El modelo transformado solo acepta relaciones binarias, entonces una ternaria no se puede quedar así. Lo que se hace es cambiarla por una entidad que la represente, y conectar esa entidad con cada uno de los tres participantes con relaciones 1:N. El atributo *cantidad* se va para adentro de la entidad nueva, porque depende de los tres al tiempo.

La clave de ABASTECER es **nit + codigo_servicio + fecha_arribo + codigo_suministro**.

Por qué toca meter los identificadores de las tres entidades

Porque la cardinalidad es M:N:N, y eso quiere decir que ninguna pareja determina a la tercera. El enunciado lo dice clarito: un mismo suministro lo pueden entregar varios contratistas en la misma recalada. Entonces si tratas de identificar una entrega solo con (recalada, suministro), te van a salir varias filas distintas con esos mismos valores y no las puedes diferenciar. Necesitas las tres. Y como RECALADA es débil, ella sola mete dos pedazos en vez de uno, por eso la clave termina con cuatro atributos y no con tres.

Las tres relaciones quedan igual en cardinalidad y participación: CONTRATISTA (1) parcial — ABASTECER (N) total; RECALADA (1) parcial — ABASTECER (N) total; SUMINISTRO (1) parcial — ABASTECER (N) total.

Si solo pudiera haber un contratista por recalada y suministro, la cardinalidad cambiaría a M:N:1 del lado de CONTRATISTA, porque la pareja (recalada, suministro) de esta forma diría quién es el contratista. La clave se acortaría a **codigo_servicio + fecha_arribo + codigo_suministro**, y *nit* se saldría de la clave para quedar como un atributo normal de ABASTECER.

Modelo: 05_Ej3a_ABASTECER.drawio

3b. La agregación DEPOSITA

Una agregación es cuando coges una relación entera y la tratas como si fuera una entidad, para poder relacionarla con otra cosa. El modelo transformado no las acepta.

Lo bueno es que acá no hay que hacer casi nada, porque de esta forma lo habíamos resuelto sin darnos cuenta en el punto 2a. Como EMBARQUE dejó de ser relación y se volvió entidad, la agregación se desarma sola: ahora simplemente pegamos BODEGA contra esa entidad, como con cualquier otra. Queda apode esta formada sobre EMBARQUE, que es lo único en todo el modelo transformado que significa "este importador participando en esta recalada". Ninguna otra sirve.

- Si la colgaras de **IMPORTADOR**, estarías diciendo que cada importador usa siempre la misma bodega, sin importar en qué escala llegó su carga. Se pierde que un mismo importador puede usar bodegas distintas en viajes distintos.
- Si la colgaras de **RECALADA**, estarías diciendo que toda la carga directamente escala se va a una sola bodega. Se pierde la separación por importador cuando varios embarcan en el mismo barco, que es lo que pasa siempre en un puerto.

La cardinalidad y la participación son EMBARQUE (N) total — BODEGA (1) parcial. Todo embarque va a parar a una sola bodega, mientras que una bodega puede estar vacía en este momento y con el tiempo recibe un montón de embarques.

Un detalle que importa: *codigo_bodega* **no** entra en la clave de EMBARQUE. Es un dato descriptivo y de esta forma. EMBARQUE tenía su clave completa desde antes de que apareciera BODEGA, así que esa relación es normalita.

Modelo: 06_Ej3b_DEPOSITA.drawio

4. Especialización y relaciones recursivas

4a. La jerarquía de OPERARIO

La especialización se cambia por una relación binaria 1:1 entre OPERARIO y cada subclase. Cada subclase queda como su propia entidad, con sus atributos, y hereda *id_operario* como clave. La participación es OPERARIO (1) parcial — GRUERO (1) total, y OPERARIO (1) parcial — ESTIBADOR (1) total.

Que la jerarquía es **parcial** se ve en que la participación de OPERARIO en las dos relaciones SER es parcial, o sea línea sencilla. Eso es lo que deja que existan operarios que no son ni grueros ni estibadores, que es justo lo que dice el enunciado. Del otro lado sí va total, porque todo gruero es sí o sí un operario.

Que es **solapada** se ve en que son dos relaciones separadas y no una sola con opciones que se peleen entre sí. Como cada una va por su lado, nada impide que el mismo *id_operario* aparezca en GRUERO y en ESTIBADOR al tiempo, que es justo lo que el enunciado pide poder representar.

Qué estrategia queda descartada por el solapamiento

La de tabla única con atributo discriminante, esa donde metes todo en una sola entidad y le agregas un campo *tipo_operario* que diga si es gruero o estibador. Esa no sirve, porque un campo de un solo valor no puede decir "es las dos cosas". Te tocaría escoger una, y ahí la jerarquía se vuelve exclusiva, que es justo lo contrario de lo que necesitamos.

Y por el lado de que es parcial, tampoco sirve dejar solo las subclases y borrar la superclase, porque los operarios que no son ni grueros ni estibadores se quedarían sin ningún lado donde existir.

Y *equipos_manejados* es multivaluado, así que le aplicamos la misma regla del ejercicio 1: sale de ESTIBADOR y se vuelve la entidad EQUIPO_ESTIBADOR, pegada con una relación 1:N. Clave: **id_operario + equipo**. Participación: ESTIBADOR (1) parcial — EQUIPO_ESTIBADOR (N) total.

Modelo: 07_Ej4a_ESPECIALIZACION.drawio

4b. Las dos relaciones recursivas

De las dos, solo toca transformar SUPERVISAR. Y la razón importa: lo que obliga a transformar no es que la relación sea recursiva. Una recursiva 1:N es una relación binaria 1:N como cualquier otra y pasa derecho. Lo que decide es la cardinalidad.

- **SUPERVISAR** es M:N recursiva y trae el atributo *periodo*. Sí se transforma.
- **TRANSBORDAR** es 1:N recursiva y no tiene atributos. No se transforma.

A SUPERVISAR le hicimos lo mismo del ejercicio 2: se vuelve la entidad SUPERVISION, con dos relaciones 1:N hacia OPERARIO. Los nombres de rol **supervisor** y **supervisado** se quedan escritos sobre las líneas, que es lo que deja saber quién manda a quién. La participación es OPERARIO (1) parcial en las dos y SUPERVISION (N) total en las dos, y la clave queda **id_operario(supervisor) + id_operario(supervisado) + periodo**.

Ese *periodo* queda como atributo de SUPERVISION, pero además entra en la clave. Y eso no es opcional: el enunciado dice que un operario puede ser supervisado por distintos operarios en periodos distintos,

entonces si le sacas el *periodo* a la clave, dos supervisiones de la misma pareja en épocas diferentes quedan idénticas y el modelo no las puede separar.

TRANSBORDAR se mantiene igual, con su rombo y sus roles *recalada_origen* y *recalada_destino*, participación parcial en los dos lados.

Una aclaración que vale la pena hacer: en el modelo transformado no se dibuja la clave foránea como si fuera un atributo. La clave foránea es del modelo relacional, no del entidad–relación. Acá la relación se muestra con el rombo y sus roles, y la FK aparece después, cuando de esta forma pasemos a tablas.

Modelo: 08_Ej4b_SUPERVISAR_TRANSBORDAR.drawio

4c. Las que no hay que tocar

Estas cuatro pasan al modelo relacional tal como están:

- **IDENTIFICAR**, SERVICIO (1) – (N) RECALADA. Es binaria 1:N. Lo único que hace es pasarle *codigo_servicio* a la clave de RECALADA, que de esta forma venía débil desde el modelo original.
- **ATENDER**, BUQUE (1) – (N) RECALADA. Binaria 1:N y sin atributos. El identificador del buque se baja al lado N y de esta forma.
- **CLASIFICAR**, BUQUE (1) – (1) CERTIFICADO. Es 1:1, se resuelve pasando la clave de un lado al otro. No toca crear nada.
- **TRANSBORDAR**, RECALADA (1) – (N) RECALADA. Es recursiva, pero es 1:N y no tiene atributos. Como dijimos arriba, ser recursiva sola no obliga a nada.

El criterio en una línea: pasa sin cambios toda relación binaria 1:1 o 1:N que no tenga atributos propios, porque eso es justo lo que el modelo transformado permite. Se implementa pasando una clave foránea y queda resuelto, sin entidades nuevas.

Todas las demás sí se transforman: EMBARQUE, CUADRILLA y SUPERVISAR por ser N:M; ABASTECER por ser ternaria; DEPOSITA por ser agregación; y la jerarquía GRUERO / ESTIBADOR por ser especialización.

5. Revisión del modelo del compañero

Le encontramos tres errores al modelo que entregó el estudiante.

Error 1. Las sedes puestas como columnas numeradas

```
CONTRATISTA (nit, razon_social, ciudad_sede1, direccion_sede1,
             ciudad_sede2, direccion_sede2)
```

Regla que se omitió. La de atributo multivaluado. Un multivaluado se saca a su propia entidad; no se arregla repitiendo el campo con un numerito al final.

Problema que genera. Deja el máximo clavado en dos sedes. Un contratista con tres no cabe, y meterlo obligaría a cambiar el esquema. Aparte, los contratistas con una sola sede quedan con la mitad de las columnas vacías, y para responder algo tan sencillo como "qué contratistas operan en Cali" toca revisar dos grupos de campos por aparte.

Forma de corregirlo. Crear la entidad SEDE_CONTRATISTA (ciudad, direccion) con relación 1:N desde CONTRATISTA. Clave: nit + ciudad + direccion. Participación: CONTRATISTA (1) parcial — SEDE_CONTRATISTA (N) total. Es exactamente lo del punto 1b.

Error 2. peso_toneladas y tipo_carga metidos en IMPORTADOR

```
IMPORTADOR (documento, nombres, apellidos, correo,
            peso_toneladas, tipo_carga)
```

Regla que se omitió. Acá se saltó dos directamente. La primera: los atributos directamente relación N:M van en la entidad que la reemplaza, nunca en uno de los extremos. La segunda, más obvia todavía: *correo* sigue ahí como multivaluado, sin transformar.

Problema que genera. Con esos campos en IMPORTADOR, cada importador tendría un solo peso y un solo tipo de carga para toda su historia con el puerto. Se pierde el detalle de cada embarque y de esta forma no se puede registrar dos envíos distintos del mismo cliente. Y lo de *correo* deja en el modelo justo el tipo de atributo que el enunciado prohíbe.

Forma de corregirlo. Mover *peso_toneladas* y *tipo_carga* a la entidad EMBARQUE, y sacar *correo* a la entidad CORREO_IMPORTADOR.

Error 3. EMBARQUE quedó desconectada de RECALADA

```
EMBARQUE (id_embarque, fecha_solicitud)
relación: IMPORTADOR (1) — (N) EMBARQUE
relación: RECALADA (N) — (1) BODEGA
```

Regla que se omitió. La entidad que reemplaza una N:M tiene que pegarse a las dos entidades originales, y su clave tiene que ser la unión de los dos identificadores. Acá le inventaron una clave artificial (*id_embarque*) y la dejaron colgando de un solo lado. Y de paso, la agregación DEPOSITA quedó dibujada contra RECALADA en vez de contra EMBARQUE.

Problema que genera. El modelo de esta forma no dice en qué recalada se embarca la carga, o sea que la relación N:M original desapareció del todo. Y al colgar BODEGA de RECALADA, toda la carga directamente

escala tendría que irse a la misma bodega sin importar cuántos importadores hade esta forman embarcado, que es justo el error que explicamos en el punto 3b.

Forma de corregirlo. Conectar EMBARQUE con RECALADA por una relación 1:N, usar como clave documento + codigo_servicio + fecha_arribo, y apode esta formar ALMACENAR sobre EMBARQUE.

Una cosa más, aunque no se incluye entre los tres errores principales: la entidad TRANSITO también sobra. TRANSBORDAR es 1:N y no tiene atributos, así que no es necesario volverla entidad. Bastaba con dejar la relación recursiva con sus roles.

Modelos en Drive
Ejercicio 1a: Abrir modelo en Drive
Ejercicio 1b: Abrir modelo en Drive
Ejercicio 2a: Abrir modelo en Drive
Ejercicio 2b: Abrir modelo en Drive
Ejercicio 3a: Abrir modelo en Drive
Ejercicio 3b: Abrir modelo en Drive
Ejercicio 4a: Abrir modelo en Drive
Ejercicio 4b: Abrir modelo en Drive